Lecture 6 – Lab 6

Searching Algorithms: Linear VS Binary

© 2026 Ahmed Elrefaiy

Choose Correct Answer

- Number of steps to find **element 11** using **<u>linear search</u>** =
  a. 2
  b. 4
  c. 5
  d. 6
- Number of steps to find **element 11** using **<u>Binary search</u>** =
  a. 2
  b. 4
  c. 5
  d. 6

Choose Correct Answer

- Number of steps to find **element 17** using **<u>linear search</u>** =
  a. 3
  b. 5
  c. 6
  d. 7
- Number of steps to find **element 17** using **<u>Binary search</u>** =
  a. 2
  b. 3
  c. 6
  d. 7

Answer the following questions

- Worst case of Linear search = ……
- Best case of Linear search = ……
- Average case of Linear search = ……
- Space complexity for Linear search = ……

- Worst case of Binary search (recursive version) = ……
- Best case of Binary search (recursive version) = ……
- Average case of Binary search (recursive version) = ……

- Worst case of Binary search (Iterative version) = ……
- Best case of Binary search (Iterative version) = ……
- Average case of Binary search (Iterative version) = ……

- Space complexity for Binary search (Iterative version) = ……
- Space complexity for Binary search (recursive version) = ……

Write and implement the following Algorithms

- Linear search
- Binary search (iterative version)
- Binary search (recursive version)
- Write **recursive version** algorithm of **Linear search**? And find its Time and Space complexity?

The Binary Search Diagnostic Test

Not sure if a complex problem can be optimized? Run it through this two-step diagnostic flowchart.

Step 1: Can you convert the problem into a True/False question for each individual item?

↓ **Yes**

Step 2: Is the resulting T/F sequence Monotonic? (Are all Fs grouped together, and all Ts grouped together, with only ONE transition?)

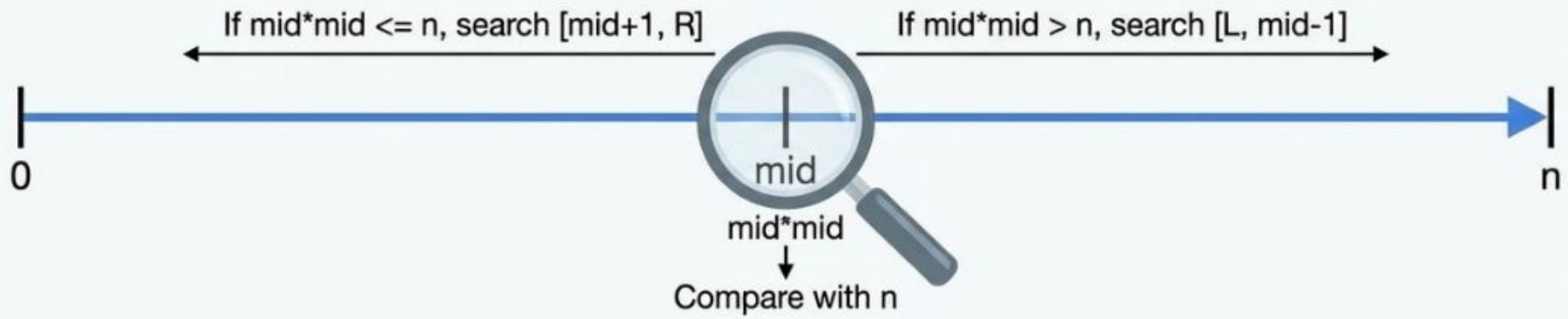
↓ **Yes**

Binary Search Will Work!

If your condition maps to an array like [F, F, F, T, T, T], you can use **Binary Search** to find that exact transition point in O(log n) time.

Advanced Task: Find Square Root of n using Binary Search on the Answer Space

The idea: The square root of ‘n’ must lie within the range [0, n]. We can binary search this range to find the largest integer ‘x’ such that x*x <= n.



The Range

The answer space is the continuous sorted range [0, n].

The Condition

We define a custom function `isValid(mid)` which checks if `mid * mid <= n`. This returns True or False.

The Logic

If `isValid(mid)` is True, the square root is at `mid` or to its right. We store `mid` and continue searching the right half. If False, we search the left half.

Write and Implement the following algorithms

1. Find (Lower Bound) the index of the first element >= target (insertion point).
2. Find (Upper Bound) the index of the first element > target.
3. Find the index of the first occurrence of target
4. Find the index of the last occurrence of target?

5. **[Your Task]** write algorithm and implement the following algorithm:
   - **Find square root of n using binary search on the answer space**